



CS1004 – Object Oriented Programming

Lecture # 25
Monday, May 23, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf

Today's Lecture

- Generic Programming
 - Templates
 - Function Templates
 - Class Templates

Lets motivate ourselves

- Number of algorithms are data-type independent
 - sorting
 - searching
 - finding n-th largest
 - swapping etc.
- Write once, use many times philosophy
 - dramatically reduces LOC (lines-of-code)
 - makes code much easier to maintain

In C++

- Templates = generic programming
- Templates are powerful features of C++ which allows you to write generic programs. In simple terms, you can create a single function or a class to work with different data types using templates.
- Two types
 - Function templates
 - Special functions that can operate with generic types.
 - Class templates
 - Can have members that use template parameters as types

Motivation

- A function for finding minimum value

```
int min(int a, int b) {  
    return (a < b ? a : b)  
}
```

- For floats we will need another function

```
float min(float a, float b) {  
    return (a < b ? a : b)  
}
```

- Can we parameterize this function with a type?

- Towards Generic Programming
- Like objects are generalized into classes, we would like to generalize classes and functions into templates

Motivation

```
void swap(int& a, int& b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}
```

```
void swap(float& a, float& b) {  
    float temp = a;  
    a = b;  
    b = temp;  
}
```

```
void swap(double& a, double& b) {  
    double temp = a;  
    a = b;  
    b = temp;  
}
```

```
void swap(char& a, char& b) {  
    char temp = a;  
    a = b;  
    b = temp;  
}
```

```
void swap(string& a, string& b) {  
    string temp = a;  
    a = b;  
    b = temp;  
}
```

```
void swap(Time& a, Time& b) {  
    Time temp = a;  
    a = b;  
    b = temp;  
}
```

```
template <class T>
T someFunction(T arg)
{
    ... ..
}
```

- In the above code, T is a template argument that accepts different data types (int, float), and class is a keyword
- You can also use keyword **typename** instead of class in the above example
- When, an argument of a data type is passed to someFunction(), compiler generates a new version of someFunction() for the given data type

Templates

- C++ provides a **super macro** mechanism that is identified by the keyword **template**
- **Templates** can be thought of as MACROs for functions and classes, that need not be explicitly called by the programmer
- **Templates** are controlled by the **compiler** (and not by the pre-processor)
- Arguments that are passed to template functions are checked for type-mismatch

Function Template

- Preserving the semantics of a function
- **Automatically generated** instance of a function, varying by type
- The programmer parameterizes all or a subset of the types in the interface
- The function implementation remains invariant over a set of function instances, each handling a unique data type

```
template <class Type>  
Type min(Type a, Type b) {  
    return (a>b ? a : b);  
}
```

Template Mechanism

- Declare a type parameter (type placeholder) and use it in the function instead of a specific type. This requires a different kind of parameter list:

```
template <typename DataType > // type parameter
void Swap(____ &first, ____ & second) DataType
{
    DataType temp = first;
    first = second;
    second = temp;
}
```

Templates

```
template <class T>
T min(T a, T b) {
return (a<b ? a : b)
}
```

```
int main()
{
int a = 3, b = 4;
float c = 2.5, d = 4.5;
cout << min(a, b); // This will generate a function with int
cout << min(c, d); // This will generate a function with float
return 0;
}
```

Template Instantiation

- A function template is only a pattern that describes how individual functions can be built from given actual types. This process of constructing a function is called **instantiation**
- We instantiated **Swap()** many times — with types **int**, **double**, **string**, and **Time**. In each instantiation, the type parameter is said to be **bound** to the actual type passed
- A template thus serves as a pattern for the definition of an **unlimited number of instances**

Function Templates Cont.

- For **each call** to a function template, the compiler will try to **generate an actual function** with the appropriate prototype
- **Overloading** for function of the same name is done in this order:
 - Look for an exact match among non-template functions
 - Look for an exact match that can be generated from function template
 - Look for the best match non-template functions
 - Special functions using template types

Function Templates Cot..

- A template parameter is a special kind of parameter used to pass a type as argument
 - Just like regular function parameters
 - But pass types to a function

Declaration Format?

- Format for declaring function templates with type parameters
- `template <class identifier> function_declaration;`
- `template <typename identifier> function_declaration;`
- Same functionality, different keywords
 - use `<typename ...>`
 - older compilers only used `<class ...>` format

- ➡ create a template function that returns the greater one of two objects

```
template <class myType>
myType GetMax(myType a, myType b) {
return (a>b ? a : b);
}

template <typename myType>
myType GetMax(myType a, myType b) {
return (a>b ? a : b);
}
```


Some words on Templates

- **template** keyword must appear before function or class templates...
- followed by the list of **generic** or **template** types
 - Can have more than one generic type
- Followed by function declaration and/or definition
 - template function uses **myType** as a type, not data

Usage

```
int main() {  
    int i = 5, j = 6, k;  
    long l = 10, m = 5, n;  
    k = GetMax(i, j);    n = GetMax(j, m);  
}
```

- ➡ One **template** type only in definition
- ➡ call mixes **int** and **long** types

Multiple template types

- ➡ Define template types after the template keyword

```
template <typename T1, typename T2>  
myType GetMax(T1 a, T2 b) {  
    return (a>b ? a : b);  
}
```

- ➡ No difference in function call

Class Templates

- ➡ The template concept can be extended to classes.

Class Templates

```
template <class T>
class className
{
    ... ..
public:
    T var;
    T someOperation(T arg);
    ... ..
};
```

- In the above declaration, **T** is the template argument which is a placeholder for the data type used.
- Inside the class body, a member variable `var` and a member function `someOperation()` are both of type **T**.

Class template

- So far the classes that we define use fix data types
- Sometime it is useful to allow storage in a class for different data types.
- Example
 - What if we want to make a simple list of `double` type?
 - Copy paste the whole file and replace `int` with `double`

Class template

- Function templates allow writing generic functions that work on many types.
- Same idea applies to defining generic classes that work with many types
- Extract the type to be a template to make a generic classes.

Class template

- To make a class into a template, prefix the class definition with the syntax:

```
template<class T>
```

- Here `T` is just a type parameter. Like a function parameter, it is a place holder.
- When the class is instantiated, `T` is replaced by a real type.
- To access a member , use the following syntax:

- `className< T >:: memberName`

- `SafeArray< T >:: addElement();`

- Using the class template:

```
className<real type> variable;
```

```
SafeArray<int> iSafeArray;
```


Example

```
template <class T>
class SafeArray {
    T arr[20];
public:
    SafeArray(){}
    void addElement(int i, T val) {
        if (i >= 0 && i < 20)
            arr[i] = val;
    }
    T getElement(int i) {
        if (i >= 0 && i < 20)
            return arr[i];
    }
};
```

```
int main()
{
    SafeArray<int> iSafeArray;
    SafeArray<float> fSafeArray;
    for (int i = 0; i < 20; ++i)
    {
        iSafeArray.addElement(i, i*i);
        fSafeArray.addElement(i, sqrt(i));
    }
    for (int i = 0; i < 20; ++i)
    {
        cout << iSafeArray.getElement(i) << ' ';
        cout << fSafeArray.getElement(i) << endl;
    }
    return 0;
}
```

template class out of line function

- ➡ If the member functions are defined externally (outside of the class specification), we need a new syntax
- ➡ The expression `template<class Type>` must precede not only the class definition, but each externally defined member function as well

Out of line function

```
template <class T>
T SafeArray<T>::getElement(int i)
{
    if (i >= 0 && i < 20)
        return arr[i];
}
```

Instantiating class templates

- To use a class template in a program/function/library:
Instantiate it by using a declaration of the form

`className<Type> object;`

- To pass `Type` as an argument to the class template definition

- Examples:

`SafeArray<int> iSafeArray;`

`SafeArray<float> fSafeArray;`

- Compiler will generate two distinct definitions of `SafeArray` — two instances — one for `ints` and one for `floats`.

Questions

